Solutions:

Q1
(a)
Prompt:
'Please generate a synthetic dataset of user-product interactions for a movie recommender system. The dataset should include the following columns: user_id, movie_id, rating (1-5), and timestamp. Generate 20 rows of data. Then, provide Python code to create a bar chart showing the average rating per movie.'

Output from GPT:

python Copy Download

```python
import pandas as pd
import matplotlib.pyplot as plt

# Synthetic dataset
data = {
    "user_id": [1, 1, 2, 2, 3, 3, 4, 4, 5, 5, 6, 6, 7, 7, 8, 8, 9, 9, 10, 10],
    "movie_id": [101, 102, 101, 103, 102, 104, 101, 105, 103, 104, 105, 101, 102, 103, 104,
105, 101, 102, 103, 104],
    "rating": [5, 4, 3, 5, 4, 2, 5, 3, 4, 5, 3, 4, 5, 2, 4, 3, 5, 4, 3, 5],
    "timestamp": ["2023-01-01", "2023-01-02", "2023-01-03", "2023-01-04", "2023-01-05",
                  "2023-01-06", "2023-01-07", "2023-01-08", "2023-01-09", "2023-01-10",
                  "2023-01-11", "2023-01-12", "2023-01-13", "2023-01-14", "2023-01-15",
                  "2023-01-16", "2023-01-17", "2023-01-18", "2023-01-19", "2023-01-20"]
}

df = pd.DataFrame(data)

# Calculate average rating per movie
avg_ratings = df.groupby('movie_id')['rating'].mean()

# Plot
plt.bar(avg_ratings.index, avg_ratings.values)
plt.xlabel('Movie ID')
plt.ylabel('Average Rating')
plt.title('Average Rating per Movie')
plt.show()
```

(b)
Prompt:
'I have a sample from the MovieLens-1M dataset with columns: user_id, movie_id, rating, timestamp, gender, age, occupation. Some ratings are missing. Write Python code to:
1. Load the data from a CSV file.
2. Handle missing values in the 'rating' column by replacing them with the median rating.
3. Encode the 'gender' column using label encoding and the 'occupation' column using one-hot encoding.'

Output from GPT:

```python
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Load dataset
df = pd.read_csv('movielens_sample.csv')

# Handle missing values in 'rating'
median_rating = df['rating'].median()
df['rating'].fillna(median_rating, inplace=True)

# Encode 'gender' using label encoding
le = LabelEncoder()
df['gender_encoded'] = le.fit_transform(df['gender'])

# Encode 'occupation' using one-hot encoding
df = pd.get_dummies(df, columns=['occupation'])

print(df.head())
```

(c)
Prompt:
'Write a Python script to preprocess the MovieLens-1M dataset for a collaborative filtering model using the Surprise library. The script should:
1. Load the dataset from a CSV file with columns: user_id, movie_id, rating, timestamp.
2. Convert the dataset into a Surprise Dataset format.
3. Split the data into train and test sets.
4. Display the first 5 processed entries from the training set.'

GPT generated code with errors:

```python
from surprise import Dataset, Reader
from surprise.model_selection import train_test_split
import pandas as pd

# Load data
df = pd.read_csv('movielens-1m.csv')

# Define reader
reader = Reader(rating_scale=(1, 5))

# Load dataset into Surprise
data = Dataset.load_from_df(df[['user_id', 'movie_id', 'rating']], reader)

# Split data
trainset, testset = train_test_split(data, test_size=0.25)

# Show first 5 training entries
for i, (uid, iid, rating) in enumerate(trainset.all_ratings()):
    if i < 5:
        print(f"User: {uid}, Movie: {iid}, Rating: {rating}")
```

Corrected code:

```python
from surprise import Dataset, Reader
from surprise.model_selection import train_test_split
import pandas as pd

# Load data
df = pd.read_csv('movielens-1m.csv')

# Define reader
reader = Reader(rating_scale=(1, 5))

# Load dataset into Surprise
data = Dataset.load_from_df(df[['user_id', 'movie_id', 'rating']], reader)

# Split data
trainset, testset = train_test_split(data, test_size=0.25, random_state=42)

# Convert trainset to list of tuples for inspection
train_entries = []
for uid, iid, rating in trainset.all_ratings():
    train_entries.append((uid, iid, rating))

# Show first 5 training entries
for i in range(5):
    print(f"User: {train_entries[i][0]}, Movie: {train_entries[i][1]}, Rating: {train_entri
es[i][2]}")
```

Q3
(a)

```sql
SELECT
    h.hacker_id,
    h.name,
    COUNT(DISTINCT s.task_id) AS task_count
FROM Hackers h
JOIN Submissions s ON h.hacker_id = s.hacker_id
GROUP BY h.hacker_id, h.name
HAVING COUNT(DISTINCT s.task_id) > (
    SELECT AVG(task_count)
    FROM (
        SELECT COUNT(DISTINCT task_id) AS task_count
        FROM Submissions
        GROUP BY hacker_id
    ) AS avg_tasks
)
ORDER BY task_count DESC, h.hacker_id ASC;
```

(b)

```sql
SELECT
    t.task_id,
    t.description,
    t.bonus * COUNT(*) as total_bonus
FROM Tasks t
JOIN (
    SELECT task_id, hacker_id
    FROM Submissions s1
    WHERE (
        SELECT COUNT(*)
        FROM Submissions s2
        WHERE s2.task_id = s1.task_id
        AND (s2.score > s1.score OR (s2.score = s1.score AND s2.submission_id < s1.submis
sion_id))
    ) < 3
) AS top3 ON t.task_id = top3.task_id
GROUP BY t.task_id, t.description, t.bonus
ORDER BY t.task_id ASC;
```

(c)

```sql
SELECT
    s.submission_id,
    s.hacker_id,
    h.name,
    s.score
FROM Submissions s
JOIN Hackers h ON s.hacker_id = h.hacker_id
WHERE s.submission_date = '2023-01-01'
AND s.score = (
    SELECT MAX(score)
    FROM Submissions s2
    WHERE s2.task_id = s.task_id
    AND s2.submission_date = '2023-01-01'
)
AND s.submission_id = (
    SELECT MIN(submission_id)
    FROM Submissions s3
    WHERE s3.task_id = s.task_id
    AND s3.submission_date = '2023-01-01'
    AND s3.score = s.score
)
ORDER BY s.task_id ASC;
```

(d)

```sql
SELECT
    h.hacker_id,
    h.name,
    COUNT(DISTINCT s.task_id) AS tasks_participated,
    COALESCE(SUM(s.best_score), 0) AS total_score,
    ROUND(AVG(s.best_score), 2) AS average_score
FROM Hackers h
LEFT JOIN (
    SELECT
        hacker_id,
        task_id,
        MAX(score) AS best_score
    FROM Submissions
    GROUP BY hacker_id, task_id
) s ON h.hacker_id = s.hacker_id
GROUP BY h.hacker_id, h.name
ORDER BY total_score DESC, h.hacker_id ASC;
```

(e)

```sql
SELECT
    h.hacker_id,
    h.name,
    h.bank_account
FROM Hackers h
LEFT JOIN Submissions s ON h.hacker_id = s.hacker_id
WHERE s.hacker_id IS NULL;
```

Q4

(a) 1. **Collaborative filtering**: Finds a subset of users who have similar tastes and preferences to the target user and uses this subset for offering recommendations. Basic assumptions are that users with similar interests have common preferences and sufficient user preferences are available.
2.**Matrix factorization**: Represents each item and user as an embedding vector and calculates their preference score (e.g., inner product). It factorizes the user-item interaction matrix into user and item matrices.

(b) Cold start: Occurs when there is insufficient data about new users or items to make accurate recommendations. This affects recommendation quality because the system cannot find similar users/ items or generate meaningful embeddings.

Mitigate strategies:
Content-based filtering: Use item features (genre, actors, etc.) or user demographics to make initial recommendations

1. **Hybrid approaches**: Combine collaborative filtering with content-based methods
2. **Random exploration**: Initially recommend diverse items to collect preference data
3. **Use of default ratings**: Apply average or default ratings for new users/items

(c) Guidance:

1. Choose and implement one collaborative filtering approach
2. Calculate similarity scores (cosine, Pearson, etc.) as shown in lecture notes
3. Generate top-10 recommendations for each user
4. Evaluate using Hit Rate and F1 score formulas from the lecture notes

Rubric:

- **Code implementation** (8 marks):
  - Correct data loading and preprocessing (2 marks)
  - Proper implementation of chosen algorithm (3 marks)
  - Correct train-test split (1 mark)
  - Accurate top-N recommendation generation (2 marks)
- **Evaluation metrics** (4 marks):
  - Correct Hit Rate calculation (2 marks)
  - Correct F1 score calculation (2 marks)
- **Documentation** (2 marks):
  - Clear explanation of approach (1 mark)
  - Discussion of results (1 mark)